```python
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
import torch
import torch.nn as nn

print("TensorFlow:", tf.__version__, "| PyTorch:", torch.__version__)
```

```
TensorFlow: 2.21.0 | PyTorch: 2.11.0+cpu
```

```python
# TensorFlow
a_tf = tf.constant([[1., 2.], [3., 4.]])
b_tf = tf.constant([[5., 6.], [7., 8.]])
print(a_tf + b_tf)                      # element-wise add
print(tf.matmul(a_tf, b_tf))            # matrix multiply: [[19, 22],
[43, 50]]
print(tf.reshape(a_tf, (4,)), a_tf.shape, a_tf.dtype)
print(a_tf.numpy())                     # convert to numpy

# PyTorch
a_pt = torch.tensor([[1., 2.], [3., 4.]])
b_pt = torch.tensor([[5., 6.], [7., 8.]])
print(a_pt + b_pt)
print(a_pt @ b_pt)                      # matrix multiply (same as
torch.matmul)
print(a_pt.reshape(4), a_pt.shape, a_pt.dtype)
print(a_pt.numpy())
```

```
tf.Tensor(
[[ 6.  8.]
 [10. 12.]], shape=(2, 2), dtype=float32)
tf.Tensor(
[[19. 22.]
 [43. 50.]], shape=(2, 2), dtype=float32)
tf.Tensor([1. 2. 3. 4.], shape=(4,), dtype=float32) (2, 2) <dtype:
'float32'>
[[1. 2.]
 [3. 4.]]
tensor([[ 6.,  8.],
        [10., 12.]])
tensor([[19., 22.],
        [43., 50.]])
tensor([1., 2., 3., 4.]) torch.Size([2, 2]) torch.float32
[[1. 2.]
 [3. 4.]]
```

```python
# TensorFlow: record operations on a "tape", then ask for the gradient
x = tf.Variable(2.0)
with tf.GradientTape() as tape:
    y = x ** 2 + 3 * x
```

```
print("TensorFlow dy/dx:", tape.gradient(y, x).numpy())      # 7.0

# PyTorch: mark the tensor, call backward(), read .grad
x = torch.tensor(2.0, requires_grad=True)
y = x ** 2 + 3 * x
y.backward()
print("PyTorch dy/dx:", x.grad.item())                       # 7.0
```

```
TensorFlow dy/dx: 7.0
PyTorch dy/dx: 7.0
```

```
np.random.seed(42)
X = np.random.rand(200, 1).astype("float32")
y = (3 * X + 2 + 0.1 * np.random.randn(200, 1)).astype("float32")

tf_model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(1,)),
    tf.keras.layers.Dense(1),                        # one neuron:
output = w*x + b
])
tf_model.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=0.1),
loss="mse")
hist = tf_model.fit(X, y, epochs=1000, batch_size=200, verbose=0)
# batch of 200 = all the data at once

w, b = tf_model.get_weights()
print("TensorFlow -> weight:", w[0][0], "| bias:", b[0], "| final
loss:", hist.history["loss"][-1])
```

```
TensorFlow -> weight: 2.9921834 | bias: 2.0105252 | final loss:
0.009338258765637875
```

```
Xt, yt = torch.from_numpy(X), torch.from_numpy(y)

pt_model = nn.Linear(1, 1)                                    # one
neuron: output = w*x + b
loss_fn = nn.MSELoss()
optimizer = torch.optim.SGD(pt_model.parameters(), lr=0.1)

pt_losses = []
for epoch in range(1000):
    pred = pt_model(Xt)                  # forward pass
    loss = loss_fn(pred, yt)             # compute the loss
    optimizer.zero_grad()                # clear old gradients
    loss.backward()                      # compute new gradients
    optimizer.step()                     # update the weights
    pt_losses.append(loss.item())

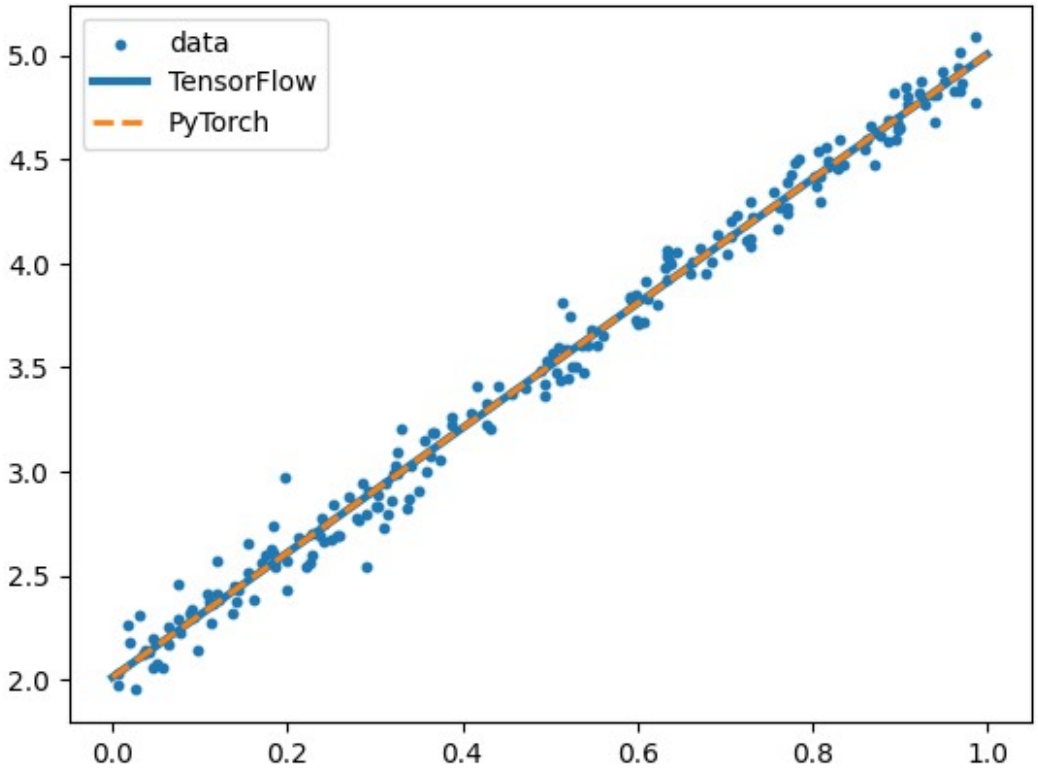
print("PyTorch    -> weight:", pt_model.weight.item(), "| bias:",
pt_model.bias.item(), "| final loss:", pt_losses[-1])
```

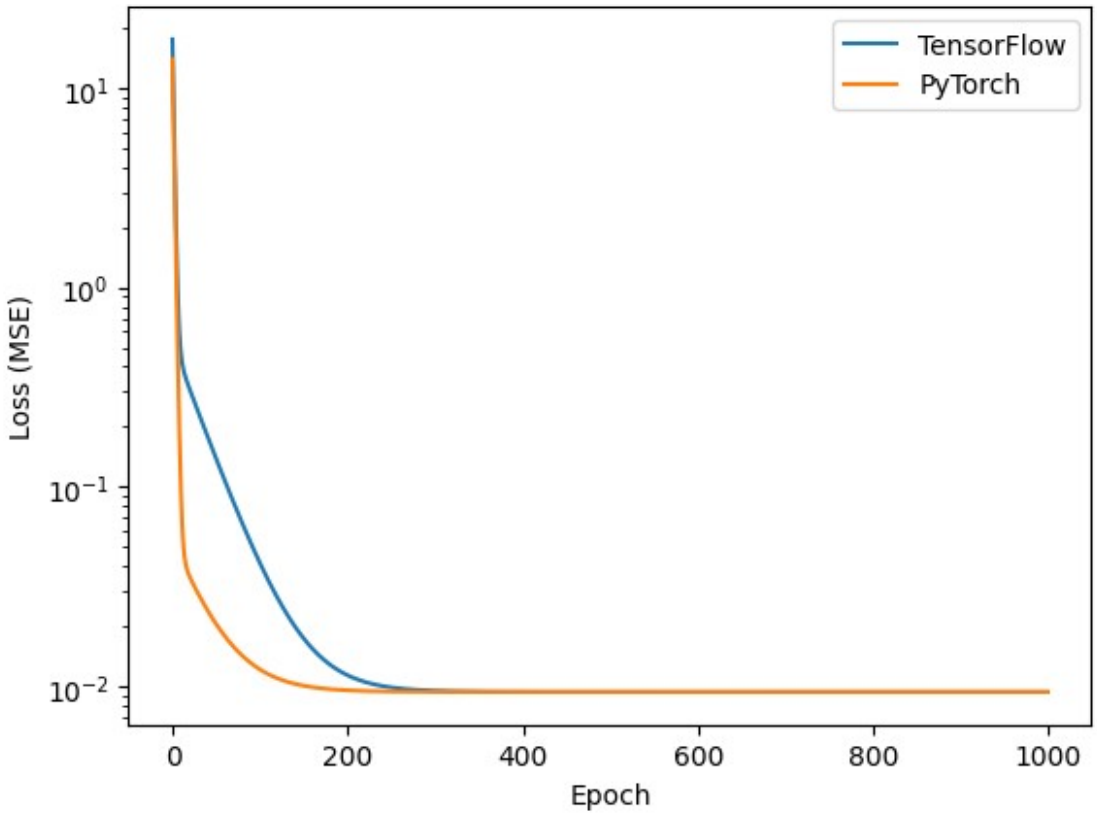
```
PyTorch    -> weight: 2.9921834468841553 | bias: 2.0105252265930176 |
final loss: 0.0093382578343153

xs = np.linspace(0, 1, 50, dtype="float32").reshape(-1, 1)

plt.scatter(X, y, s=10, label="data")
plt.plot(xs, tf_model.predict(xs, verbose=0), linewidth=3,
label="TensorFlow")
with torch.no_grad():
    plt.plot(xs, pt_model(torch.from_numpy(xs)).numpy(), "--",
linewidth=2, label="PyTorch")
plt.legend()
plt.show()

plt.plot(hist.history["loss"], label="TensorFlow")
plt.plot(pt_losses, label="PyTorch")
plt.yscale("log")
plt.xlabel("Epoch")
plt.ylabel("Loss (MSE)")
plt.legend()
plt.show()
```




TensorFlow
PyTorch
Loss (MSE)
10^{1}
10^{0}
10^{-1}
10^{-2}
0
200
400
600
800
1000
Epoch